Fed-Batch Bioreactor Kinetics with Units and Engineering Handoff

This example derives a temperature-dependent fed-batch bioreactor model from kinetic assumptions, checks units before numeric handoff, generates reusable MATLAB® functions, and reuses those functions in compact simulation, local linearization, and a rate-law handoff ready for SimBiology®.

Workflow: enzyme mechanism -> symbolic rate law -> unit audit -> fed-batch balances -> batch limit -> Jacobians -> generated functions -> simulation, state-space analysis, and SimBiology rate reuse.

Requires: MATLAB R2025a or later, Symbolic Math Toolbox™. Optional: SimBiology, Control System Toolbox™.

Copyright 2026 The MathWorks, Inc.

1. Derive the kinetic rate law from enzyme-complex assumptions

Start from the enzyme-mediated conversion $E + S \rightleftharpoons C \rightarrow E + P$, where C is the enzyme-substrate complex. The quasi-steady-state approximation sets $dC/dt = 0$ and enzyme conservation sets $E_T = E + C$.

```
syms kForward kReverse kcat enzymeFree substrate enzymeComplex enzymeTotal positive
qssaEquation = kForward*enzymeFree*substrate - kReverse*enzymeComplex - kcat*enzymeComplex == 0;
enzymeConservation = enzymeTotal == enzymeFree + enzymeComplex;
kineticSolution = solve([qssaEquation, enzymeConservation], [enzymeFree, enzymeComplex]);
complexQSSA = simplify(kineticSolution.enzymeComplex);
rateDerived = simplify(kcat*complexQSSA);
syms Vmax Km positive
rateCanonical = Vmax*substrate/(Km + substrate);
rateDefinitionCheck = simplify(subs(rateCanonical, ...
    [Vmax, Km], ...
    [kcat*enzymeTotal, (kReverse + kcat)/kForward]) - rateDerived)
```

```
rateDefinitionCheck = 0
```

The check returns zero, so the symbolic derivation recovers the Michaelis-Menten form with $V_{max} = k_{cat}E_T$ and $K_m = (k_{-1} + k_{cat})/k_1$.

The fed-batch model below reuses this saturation structure as a Monod-style specific growth law. It carries the first-principles saturation result into a microbial growth model without implying that whole-cell growth is literally a single-enzyme reaction.

Enzyme derivation	Bioreactor growth model	Interpretation
`substrate`	`S`	Limiting substrate concentration
`Vmax`	`muRef*Arrhenius(T)`	Maximum/reference specific growth rate adjusted for temperature
`Km`	`Ks`	Half-saturation constant for substrate-limited growth

2. Add temperature dependence and inspect sensitivity

With that mapping, use an apparent Arrhenius correction around a reference temperature. This is credible over a narrow calibrated range, not as a universal high-temperature growth model.

```
syms X S V T muRef Ks Ea R Tref Sf Yxs Ypx real positive
syms P F real
specificGrowthRate = muRef*exp(-(Ea/R)*(1/T - 1/Tref))*S/(Ks + S);
temperatureSensitivity = simplify(diff(specificGrowthRate, T))
```

```
temperatureSensitivity =
```

$$\frac{\mathrm{Ea}\, S\, \mathrm{muRef}\, e^{-\frac{\mathrm{Ea}\left(\frac{1}{T}-\frac{1}{\mathrm{Tref}}\right)}{R}}}{R\, T^2\, (\mathrm{Ks}+S)}$$

```
isMonotoneInTemperature = isAlways(temperatureSensitivity > 0)
```

```
isMonotoneInTemperature =
logical
   1
```

The derivative is positive under the stated assumptions. An Arrhenius-only model will push a temperature optimization to the upper allowed bound unless biology, data, or operating policy adds a thermal optimum or deactivation term.

3. Audit units before creating numeric functions

Symbolic units are regular symbolic expressions that can be converted, simplified, checked, and separated before a downstream solver sees numeric magnitudes. This section uses common lab-scale bioprocess units and includes the Celsius absolute-temperature conversion that is easy to get wrong.

```
u = symunit;
flowWithUnits = 2*u.ml/u.min;
volumeInitialWithUnits = 2*u.L;
temperatureWithUnits = 33*u.Celsius;
temperatureReferenceWithUnits = 30*u.Celsius;
activationEnergyWithUnits = 55*u.kJ/u.mol;
gasConstantWithUnits = 8.314462618*u.J/(u.mol*u.K);
temperatureAsDifference = unitConvert(temperatureWithUnits, u.K);
temperatureAsAbsolute = unitConvert(temperatureWithUnits, u.K, 'Temperature', 'absolute');
temperatureReferenceAbsolute = unitConvert(temperatureReferenceWithUnits, u.K, 'Temperature', 'absolute');
flowLitersPerHour = unitConvert(flowWithUnits, u.L/u.hr);
dilutionRateWithUnits = unitConvert(flowLitersPerHour/volumeInitialWithUnits, 1/u.hr)
```

```
dilutionRateWithUnits =
```

$$\frac{3}{50}\ \frac{1}{\mathrm{h}}$$

```
arrheniusExponentWithUnits = unitConvert( ...
    (activationEnergyWithUnits/gasConstantWithUnits)*(1/temperatureAsAbsolute - 1/temperatureReferenceAbsolute),
...
    'SI')
```

```
arrheniusExponentWithUnits =
```

$$-\frac{619244948763443200000}{2896039958997660350811}$$

```
temperatureConversionSummary = [temperatureAsDifference; temperatureAsAbsolute]
```

```
temperatureConversionSummary =
```

$$\begin{bmatrix} 33\ \mathrm{K} \\ \frac{6123}{20}\ \mathrm{K} \end{bmatrix}$$

MATLAB treats Celsius values as temperature differences unless `Temperature` is set to `absolute`. The Arrhenius exponent is dimensionless after conversion, which is required before evaluating `exp`.

4. Verify the fed-batch balance dimensions

Use unit-bearing placeholders to verify every state derivative has the expected dimension. MATLAB assumes symbolic variables are dimensionless, so the units must be attached explicitly in this audit expression.

The vector below is a unit-only copy of the fed-batch material balances that are derived symbolically in Section 6. It is written here with placeholder magnitudes such as `muMag`, `dMag`, and `xMag` so the unit check focuses only on dimensions.

Row	Balance	Physical meaning
`dXdt`	`mu*X - D*X`	Biomass grows at specific rate `mu`; feed dilution reduces cell concentration.
`dSdt`	`D*(Sf - S) - mu*X/Yxs`	Feed adds concentrated substrate and dilution removes broth substrate; cells consume substrate to make biomass.
`dPdt`	`Ypx*mu*X - D*P`	Product forms in proportion to growth; feed dilution reduces product concentration.
`dVdt`	`F`	Reactor volume increases at the inlet feed flow rate.

```
syms muMag dMag xMag sMag pMag fMag sfMag yxsMag ypxMag positive
rhsUnitAudit = [(muMag/u.hr)*(xMag*u.g/u.L) - (dMag/u.hr)*(xMag*u.g/u.L);
    (dMag/u.hr)*((sfMag*u.g/u.L) - (sMag*u.g/u.L)) - ((muMag/u.hr)*(xMag*u.g/u.L))/yxsMag;
    ypxMag*(muMag/u.hr)*(xMag*u.g/u.L) - (dMag/u.hr)*(pMag*u.g/u.L);
    fMag*u.L/u.hr];
expectedDerivativeUnits = [u.g/(u.L*u.hr); u.g/(u.L*u.hr); u.g/(u.L*u.hr); u.L/u.hr];
unitAudit = checkUnits(rhsUnitAudit == expectedDerivativeUnits);
unitAuditTable = table(["dXdt"; "dSdt"; "dPdt"; "dVdt"], ...
    unitAudit.Compatible, unitAudit.Consistent, ...
    'VariableNames', {'Derivative', 'Compatible', 'Consistent'})
```

`unitAuditTable = 4×3 table`

	Derivative	Compatible	Consistent
1	"dXdt"	true	true
2	"dSdt"	true	true
3	"dPdt"	true	true
4	"dVdt"	true	true

The generated functions use numeric magnitudes with time in hours, volume in liters, concentrations in g/L, and temperature in kelvin.

5. Convert realistic operating parameters to numeric magnitudes

These values describe a generic glucose-limited lab fed-batch fermentation. They are realistic enough for a process-dynamics example without claiming organism-specific fidelity.

```
biomassInitialWithUnits = 0.3*u.g/u.L;
substrateInitialWithUnits = 8*u.g/u.L;
productInitialWithUnits = 0*u.g/u.L;
feedSubstrateWithUnits = 250*u.g/u.L;
specificGrowthRefWithUnits = 0.32/u.hr;
halfSaturationWithUnits = 0.25*u.g/u.L;
[FVal, ~] = separateUnits(flowLitersPerHour);
[X0Val, ~] = separateUnits(biomassInitialWithUnits);
[S0Val, ~] = separateUnits(substrateInitialWithUnits);
[P0Val, ~] = separateUnits(productInitialWithUnits);
[V0Val, ~] = separateUnits(volumeInitialWithUnits);
[SfVal, ~] = separateUnits(feedSubstrateWithUnits);
[muRefVal, ~] = separateUnits(specificGrowthRefWithUnits);
[KsVal, ~] = separateUnits(halfSaturationWithUnits);
[TVal, ~] = separateUnits(temperatureAsAbsolute);
[TrefVal, ~] = separateUnits(temperatureReferenceAbsolute);
[EaVal, ~] = separateUnits(unitConvert(activationEnergyWithUnits, u.J/u.mol));
[RVal, ~] = separateUnits(gasConstantWithUnits);
YxsVal = 0.45;
YpxVal = 0.10;
initialStateVal = double([X0Val; S0Val; P0Val; V0Val]);
inputVal = double([FVal; TVal]);
paramVal = double([muRefVal; KsVal; EaVal; RVal; TrefVal; SfVal; YxsVal; YpxVal]);
parameterSummary = table(["muRef"; "Ks"; "Ea"; "R"; "Tref"; "Sf"; "Yxs"; "Ypx"], ...
    paramVal, ...
    ["1/hr"; "g/L"; "J/mol"; "J/(mol*K)"; "K"; "g/L"; "gX/gS"; "gP/gX"], ...
    'VariableNames', {'Parameter', 'Value', 'Units'})
```

parameterSummary = 8×3 table

	Parameter	Value	Units
1	"muRef"	0.3200	"1/hr"

	Parameter	Value	Units
2	"Ks"	0.2500	"g/L"
3	"Ea"	55000	"J/mol"
4	"R"	8.3145	"J/(mol*K)"
5	"Tref"	303.1500	"K"
6	"Sf"	250	"g/L"
7	"Yxs"	0.4500	"gX/gS"
8	"Ypx"	0.1000	"gP/gX"

6. Build the symbolic fed-batch material balances

The state vector is $x = [X, S, P, V]^T$. The input vector is $u = [F, T]^T$. Feed flow F is retained as the physical manipulated variable, with dilution rate $D = F/V$ defined inside the model.

```
stateVars = [X; S; P; V];
inputVars = [F; T];
paramVars = [muRef; Ks; Ea; R; Tref; Sf; Yxs; Ypx];
dilutionRate = F/V;
fedBatchRHS = [specificGrowthRate*X - dilutionRate*X;
    dilutionRate*(Sf - S) - specificGrowthRate*X/Yxs;
    Ypx*specificGrowthRate*X - dilutionRate*P;
    F];
fedBatchRHS = simplify(fedBatchRHS)
```

```
fedBatchRHS =
```

$$\begin{bmatrix} \frac{S\,X\,\mathrm{muRef}\,e^{-\frac{\mathrm{Ea}\left(\frac{1}{T}-\frac{1}{\mathrm{Tref}}\right)}{R}}}{\mathrm{Ks}+S}-\frac{F\,X}{V} \\ -\frac{F\,(S-\mathrm{Sf})}{V}-\frac{S\,X\,\mathrm{muRef}\,e^{-\frac{\mathrm{Ea}\left(\frac{1}{T}-\frac{1}{\mathrm{Tref}}\right)}{R}}}{\mathrm{Yxs}\,(\mathrm{Ks}+S)} \\ \frac{S\,X\,\mathrm{Ypx}\,\mathrm{muRef}\,e^{-\frac{\mathrm{Ea}\left(\frac{1}{T}-\frac{1}{\mathrm{Tref}}\right)}{R}}}{\mathrm{Ks}+S}-\frac{F\,P}{V} \\ F \end{bmatrix}$$

Keeping this expression symbolic makes the state, input, and parameter dependencies inspectable before they are hidden inside a numerical solver.

7. Recover the batch model as a special case

Setting the feed flow to zero removes dilution and volume growth. This boundary case is a useful check because a fed-batch model should contain the batch model as a limiting case.

```
batchRHS = simplify(subs(fedBatchRHS, F, 0))
```

```
batchRHS =
```

$$\begin{bmatrix} \frac{S\,X\,\mathrm{muRef}\,e^{-\frac{\mathrm{Ea}\left(\frac{1}{T}-\frac{1}{\mathrm{Tref}}\right)}{R}}}{\mathrm{Ks}+S} \\ -\frac{S\,X\,\mathrm{muRef}\,e^{-\frac{\mathrm{Ea}\left(\frac{1}{T}-\frac{1}{\mathrm{Tref}}\right)}{R}}}{\mathrm{Yxs}\,(\mathrm{Ks}+S)} \\ \frac{S\,X\,\mathrm{Ypx}\,\mathrm{muRef}\,e^{-\frac{\mathrm{Ea}\left(\frac{1}{T}-\frac{1}{\mathrm{Tref}}\right)}{R}}}{\mathrm{Ks}+S} \\ 0 \end{bmatrix}$$

```
batchCheck = simplify(batchRHS - [specificGrowthRate*X; -specificGrowthRate*X/Yxs; Ypx*specificGrowthRate*X; 0])
```

```
batchCheck =
```

$$\begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

The check returns zeros, confirming that no extra feed terms remain in the batch limit.

8. Use symbolic algebra to separate concentration dilution from product mass

Feed flow immediately dilutes product concentration, so $\partial \dot{P}/\partial F = -P/V$. The total product mass $M_P = PV$ answers a different question; differentiating PV with the chain rule shows the dilution terms cancel.

```
productMass = P*V;
productMassRate = simplify(jacobian(productMass, stateVars)*fedBatchRHS)
```

```
productMassRate =
```

$$\frac{S\ V\ X\ \mathrm{Ypx}\ \mathrm{muRef}\ \mathrm{e}^{-\frac{\mathrm{Ea}\left(\frac{1}{T}-\frac{1}{\mathrm{Tref}}\right)}{R}}}{\mathrm{Ks}+S}$$

```
productConcentrationFeedSensitivity = simplify(diff(fedBatchRHS(3), F))
```

```
productConcentrationFeedSensitivity =
```

$$-\frac{P}{V}$$

```
productMassFeedSensitivity = simplify(diff(productMassRate, F))
```

```
productMassFeedSensitivity = 0
```

This is the natural iteration point. If the downstream objective is product concentration, feed appears as a direct dilution penalty. If the objective is total product mass, feed affects the trajectory indirectly through substrate and volume, not through an instantaneous dilution term.

9. Compute Jacobians for local linearization and sensitivity analysis

The same symbolic RHS gives the state and input Jacobians needed for local linear models, stiffness checks, sensitivity ODEs, and controller interpretation.

```
stateJacobian = simplify(jacobian(fedBatchRHS, stateVars));
inputJacobian = simplify(jacobian(fedBatchRHS, inputVars));
size(stateJacobian)
```

```
ans = 1×2
     4     4
```

```
size(inputJacobian)
```

```
ans = 1×2
     4     2
```

The matrices stay symbolic until a specific operating point is selected.

10. Generate reusable MATLAB functions

Use explicit argument groups so every generated function has the same calling convention. The generated functions are numeric and unit-free because the unit conversion happened upstream.

```
productivityRate = simplify(Ypx*specificGrowthRate*X);
productivityGradient = simplify(jacobian(productivityRate, [S, T]));
matlabFunction(specificGrowthRate, temperatureSensitivity, ...
    'File', 'bioreactorRates', ...
    'Vars', {S, T, muRef, Ks, Ea, R, Tref}, ...
    'Outputs', {'mu', 'dmu_dT'}, ...
    'Optimize', true);
matlabFunction(fedBatchRHS, ...
    'File', 'fedBatchBioreactorRHS', ...
    'Vars', {stateVars, inputVars, paramVars}, ...
    'Outputs', {'dstate'}, ...
    'Optimize', true);
matlabFunction(stateJacobian, inputJacobian, ...
    'File', 'bioreactorJacobian', ...
```

```
    'Vars', {stateVars, inputVars, paramVars}, ...
    'Outputs', {'A', 'B'}, ...
    'Optimize', true);
matlabFunction(productivityRate, ...
    'File', 'bioreactorProductivity', ...
    'Vars', {stateVars, inputVars, paramVars}, ...
    'Outputs', {'productivity'}, ...
    'Optimize', true);
matlabFunction(productivityGradient, ...
    'File', 'bioreactorProductivityGradient', ...
    'Vars', {stateVars, inputVars, paramVars}, ...
    'Outputs', {'gradientST'}, ...
    'Optimize', true);
generatedFiles = ["bioreactorRates.m"; "fedBatchBioreactorRHS.m"; ...
    "bioreactorJacobian.m"; "bioreactorProductivity.m"; "bioreactorProductivityGradient.m"]
```

```
generatedFiles = 5×1 string
"bioreactorRates.m"
"fedBatchBioreactorRHS.m"
"bioreactorJacobian.m"
"bioreactorProductivity.m"
"bioreactorProductivityGradient.m"
```

11. Simulate the fed-batch process using the generated RHS and Jacobian

The solver receives only numeric magnitudes. The generated Jacobian is supplied to `ode15s`, which is the same handoff pattern used in larger process simulation workflows.

```
timeSpanVal = [0 12];
odeOptions = odeset('Jacobian', @(timeVal, stateVal) bioreactorJacobian(stateVal, inputVal, paramVal), ...
    'NonNegative', 1:4);
[timeSol, stateSol] = ode15s(@(timeVal, stateVal) fedBatchBioreactorRHS(stateVal, inputVal, paramVal), ...
    timeSpanVal, initialStateVal, odeOptions);
productMassSol = stateSol(:, 3).*stateSol(:, 4);
simulationSummary = table(["X"; "S"; "P"; "V"; "P*V"], ...
    [stateSol(1, :)'; productMassSol(1)], ...
    [stateSol(end, :)'; productMassSol(end)], ...
    ["g/L"; "g/L"; "g/L"; "L"; "g"], ...
```

```
    'VariableNames', {'Quantity', 'Initial', 'Final', 'Units'})
```

simulationSummary = 5×4 table

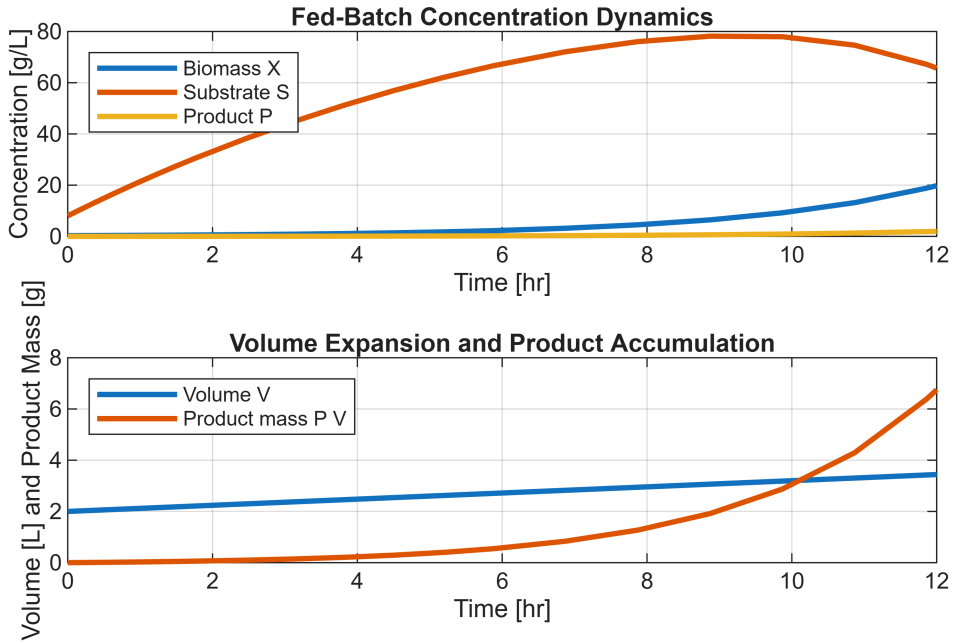
	Quantity	Initial	Final	Units
1	"X"	0.3000	19.7955	"g/L"
2	"S"	8	65.7015	"g/L"
3	"P"	0	1.9621	"g/L"
4	"V"	2	3.4400	"L"
5	"P*V"	0	6.7496	"g"

12. Visualize concentration and product-mass trajectories

Concentrations and total product mass tell different engineering stories, so plot both. The concentration traces are what a sensor or soft sensor sees; total product mass is closer to the production objective.

```
tiledlayout(2, 1)
nexttile
plot(timeSol, stateSol(:, 1), 'LineWidth', 2)
hold on
plot(timeSol, stateSol(:, 2), 'LineWidth', 2)
plot(timeSol, stateSol(:, 3), 'LineWidth', 2)
hold off
xlabel('Time [hr]')
ylabel('Concentration [g/L]')
title('Fed-Batch Concentration Dynamics')
legend('Biomass X', 'Substrate S', 'Product P', 'Location', 'northwest')
grid on
nexttile
plot(timeSol, stateSol(:, 4), 'LineWidth', 2)
hold on
plot(timeSol, productMassSol, 'LineWidth', 2)
hold off
xlabel('Time [hr]')
ylabel('Volume [L] and Product Mass [g]')
title('Volume Expansion and Product Accumulation')
```

```
legend('Volume V', 'Product mass P V', 'Location', 'northwest')
grid on
```



13. Evaluate a local perturbation model along the trajectory

A downstream controller or sensitivity analysis normally starts from a current operating condition. In fed-batch operation, the 12-hour simulated state is not a steady state because the volume and concentrations are still changing. The symbolic Jacobians still provide the local perturbation matrices A and B for small changes around that condition.

```
stateOperatingPointVal = stateSol(end, :)';
operatingDerivativeVal = fedBatchBioreactorRHS(stateOperatingPointVal, inputVal, paramVal);
[AOperatingVal, BOperatingVal] = bioreactorJacobian(stateOperatingPointVal, inputVal, paramVal);
operatingDerivativeSummary = table(["dXdt"; "dSdt"; "dPdt"; "dVdt"], ...
    operatingDerivativeVal, ...
    ["g/(L hr)"; "g/(L hr)"; "g/(L hr)"; "L/hr"], ...
    'VariableNames', {'Derivative', 'Value', 'Units'})
```

```
operatingDerivativeSummary = 4×3 table
```

	Derivative	Value	Units
1	"dXdt"	7.1245	"g/(L hr)"
2	"dSdt"	-10.9376	"g/(L hr)"
3	"dPdt"	0.7131	"g/(L hr)"
4	"dVdt"	0.1200	"L/hr"

```
eigenvaluesVal = eig(AOperatingVal);
localDynamicsSummary = table((1:numel(eigenvaluesVal))', eigenvaluesVal, ...
    'VariableNames', {'Mode', 'Eigenvalue_per_hr'})
```

localDynamicsSummary = 4×2 table

	Mode	Eigenvalue_per_hr
1	1	-0.0349
2	2	-0.0349
3	3	0.3589
4	4	0

```
inputSensitivitySummary = table(["feed flow F"; "temperature T"], ...
    BOperatingVal(3, :)', ...
    ["(g/L/hr)/(L/hr)"; "(g/L/hr)/K"], ...
    'VariableNames', {'Input', 'dPdtSensitivity', 'Units'})
```

inputSensitivitySummary = 2×3 table

	Input	dPdtSensitivity	Units
1	"feed flow F"	-0.5704	"(g/L/hr)/(L/hr)"
2	"temperature T"	0.0552	"(g/L/hr)/K"

The nonzero derivative table marks this as a trajectory linearization, not an equilibrium model. The feed-flow sensitivity of $\dot{P}$ is negative at this operating condition because concentration is diluted immediately. The total product-mass rate had zero direct feed sensitivity in Section 8, which is why selecting the right objective matters before optimization.

14. Inspect local productivity sensitivities

Use the generated productivity function and symbolic gradient to see which variables the product formation rate responds to at the simulated operating condition. The positive temperature derivative is consistent with the monotonicity result in Section 2, so this stops at sensitivity rather than pretending the Arrhenius-only model has an interior optimum.

```
productivityOperatingVal = bioreactorProductivity(stateOperatingPointVal, inputVal, paramVal);
productivityGradientOperatingVal = bioreactorProductivityGradient(stateOperatingPointVal, inputVal, paramVal);
productivitySensitivitySummary = table(productivityOperatingVal, ...
    productivityGradientOperatingVal(1), productivityGradientOperatingVal(2), ...
    'VariableNames', {'Productivity_g_per_L_hr', 'dProductivity_dS', 'dProductivity_dT'})
```

`productivitySensitivitySummary = 1×3 table`

	Productivity_g_per_L_hr	dProductivity_dS	dProductivity_dT
1	0.7815	4.5089e-05	0.0552

15. Hand off the generated Jacobian for local analysis

The generated Jacobian now becomes an engineering object. Because the linearization point is not a steady state, an absolute local model would include the affine offset $f(x_0, u_0)$ shown in Section 13. For input-output perturbation analysis around the nominal trajectory, the handoff uses $\Delta\dot{x} = A\Delta x + B\Delta u$ and $\Delta y = C\Delta x + D\Delta u$.

View the same state dynamics through product concentration P and total product mass $M_P = PV$. This keeps the control-analysis handoff focused on the symbolic result from Section 8 instead of turning the example into a controller-design walkthrough.

```
productConcentrationOutputVal = [0 0 1 0];
productMassOutputVal = [0 0 stateOperatingPointVal(4) stateOperatingPointVal(3)];
outputMatrixCVal = [productConcentrationOutputVal; productMassOutputVal];
directMatrixDVal = zeros(2, 2);
outputRateSensitivityVal = outputMatrixCVal*BOperatingVal;
outputRateSensitivitySummary = table(["Product concentration P"; "Total product mass P*V"], ...
    outputRateSensitivityVal(:, 1), outputRateSensitivityVal(:, 2), ...
    ["(g/L/hr)/(L/hr)"; "(g/hr)/(L/hr)"], ...
    ["(g/L/hr)/K"; "(g/hr)/K"], ...
    'VariableNames', {'Output', 'FeedRateSensitivity', 'TemperatureRateSensitivity', 'FeedUnits',
'TemperatureUnits'})
```

outputRateSensitivitySummary = 2×5 table

	Output	FeedRateSensitivity	TemperatureRateSensitivity	FeedUnits	TemperatureUnits
1	"Product concentration P"	-0.5704	0.0552	"(g/L/hr)/(L/hr)"	"(g/L/hr)/K"
2	"Total product mass P*V"	-2.2204e-16	0.1897	"(g/hr)/(L/hr)"	"(g/hr)/K"

```
stateDisplayNames = {'Biomass_X'; 'Substrate_S'; 'Product_P'; 'Volume_V'};
inputDisplayNames = {'FeedFlow_F'; 'Temperature_T'};
outputDisplayNames = {'ProductConcentration_P'; 'TotalProductMass_PV'};
stateSpaceHandoffSummary = table(["A"; "B"; "C"; "D"; "f(x0,u0)"], ...
    ["4 x 4"; "4 x 2"; "2 x 4"; "2 x 2"; "4 x 1"], ...
    ["State perturbation dynamics"; "Input perturbation sensitivities"; ...
    "Output definitions"; "Direct input-to-output terms"; "Affine offset for an absolute local model"], ...
    'VariableNames', {'Artifact', 'Size', 'Meaning'})
```

stateSpaceHandoffSummary = 5×3 table

	Artifact	Size	Meaning
1	"A"	"4 x 4"	"State perturbation dynamics"
2	"B"	"4 x 2"	"Input perturbation sensitivities"
3	"C"	"2 x 4"	"Output definitions"
4	"D"	"2 x 2"	"Direct input-to-output terms"
5	"f(x0,u0)"	"4 x 1"	"Affine offset for an absolute local model"

```
hasControlSystemToolbox = (exist('ss', 'file') == 2) && license('test', 'Control_Toolbox');
if hasControlSystemToolbox
    productOutputModel = ss(AOperatingVal, BOperatingVal, outputMatrixCVal, directMatrixDVal);
    productOutputModel.InputName = {'Feed flow F'; 'Temperature T'};
    productOutputModel.OutputName = {'Product concentration P'; 'Total product mass P*V'};
    productOutputModel.StateName = {'Biomass X'; 'Substrate S'; 'Product P'; 'Volume V'};
    stateSpaceObjectSummary = table("ss", numel(productOutputModel.InputName), ...
        numel(productOutputModel.OutputName), ...
        'VariableNames', {'ObjectType', 'Inputs', 'Outputs'})
else
    productOutputModel = struct('A', AOperatingVal, 'B', BOperatingVal, ...
        'C', outputMatrixCVal, 'D', directMatrixDVal, 'Offset', operatingDerivativeVal, ...
```

```
        'StateNames', {stateDisplayNames}, 'InputNames', {inputDisplayNames}, ...
        'OutputNames', {outputDisplayNames});
    stateSpaceObjectSummary = table("struct", numel(inputDisplayNames), ...
        numel(outputDisplayNames), ...
        'VariableNames', {'ObjectType', 'Inputs', 'Outputs'})
end
```

stateSpaceObjectSummary = 1×3 table

	ObjectType	Inputs	Outputs
1	"ss"	2	2

The feed-flow column shows immediate dilution of concentration, while total product mass has zero direct feed-flow rate sensitivity at the same state.

16. Validate the symbolic rate law for SimBiology reuse

A SimBiology model can reuse the same rate expression when the model quantities use concentration in g/L, time in hours, temperature in kelvin, and activation energy in J/mol. Before making any SimBiology API calls, check the exported expressions as a handoff contract in which every symbol has a downstream role, unit, and value or initial condition.

```
simbiologyUnitMap = table(["X"; "S"; "P"; "Ks"; "muRef"; "Ea"; "R"; "T"; "Tref"; "Yxs"; "Ypx"], ...
    ["species"; "species"; "species"; "parameter"; "parameter"; "parameter"; "parameter"; "parameter";
"parameter"; "parameter"; "parameter"], ...
    ["gram/liter"; "gram/liter"; "gram/liter"; "gram/liter"; "1/hour"; "joule/mole"; "joule/mole/kelvin";
"kelvin"; "kelvin"; "dimensionless"; "dimensionless"], ...
    'VariableNames', {'SimBiologyName', 'Role', 'Unit'})
```

simbiologyUnitMap = 11×3 table

	SimBiologyName	Role	Unit
1	"X"	"species"	"gram/liter"
2	"S"	"species"	"gram/liter"
3	"P"	"species"	"gram/liter"
4	"Ks"	"parameter"	"gram/liter"
5	"muRef"	"parameter"	"1/hour"
6	"Ea"	"parameter"	"joule/mole"

	SimBiologyName	Role	Unit
7	"R"	"parameter"	"joule/mole/kelvin"
8	"T"	"parameter"	"kelvin"
9	"Tref"	"parameter"	"kelvin"
10	"Yxs"	"parameter"	"dimensionless"
11	"Ypx"	"parameter"	"dimensionless"

```
simbiologySpeciesHandoff = table(["X"; "S"; "P"], ...
    initialStateVal(1:3), ...
    ["gram/liter"; "gram/liter"; "gram/liter"], ...
    'VariableNames', {'Name', 'InitialValue', 'Unit'})
```

simbiologySpeciesHandoff = 3×3 table

	Name	InitialValue	Unit
1	"X"	0.3000	"gram/liter"
2	"S"	8	"gram/liter"
3	"P"	0	"gram/liter"

```
simbiologyParameterHandoff = table(["muRef"; "Ks"; "Ea"; "R"; "T"; "Tref"; "Yxs"; "Ypx"], ...
    [paramVal(1); paramVal(2); paramVal(3); paramVal(4); inputVal(2); paramVal(5); paramVal(7); paramVal(8)], ...
    ["1/hour"; "gram/liter"; "joule/mole"; "joule/mole/kelvin"; "kelvin"; "kelvin"; "dimensionless";
"dimensionless"], ...
    'VariableNames', {'Name', 'Value', 'Unit'})
```

simbiologyParameterHandoff = 8×3 table

	Name	Value	Unit
1	"muRef"	0.3200	"1/hour"
2	"Ks"	0.2500	"gram/liter"
3	"Ea"	55000	"joule/mole"
4	"R"	8.3145	"joule/mole/kelvin"
5	"T"	306.1500	"kelvin"

	Name	Value	Unit
6	"Tref"	303.1500	"kelvin"
7	"Yxs"	0.4500	"dimensionless"
8	"Ypx"	0.1000	"dimensionless"

```
muForSimBiology = simplify(specificGrowthRate);
growthRateForSimBiology = simplify(muForSimBiology*X);
substrateUptakeRateForSimBiology = simplify(muForSimBiology*X/Yxs);
productFormationRateForSimBiology = simplify(Ypx*muForSimBiology*X);
simbiologyExpressions = [muForSimBiology; growthRateForSimBiology; ...
    substrateUptakeRateForSimBiology; productFormationRateForSimBiology];
handoffSymbolNames = string(arrayfun(@char, symvar(simbiologyExpressions), 'UniformOutput', false))';
hasUnitMapping = ismember(handoffSymbolNames, simbiologyUnitMap.SimBiologyName);
hasValueOrInitialCondition = ismember(handoffSymbolNames, ...
    [simbiologySpeciesHandoff.Name; simbiologyParameterHandoff.Name]);
simbiologyHandoffContract = table(handoffSymbolNames, hasUnitMapping, hasValueOrInitialCondition, ...
    'VariableNames', {'Symbol', 'HasUnitMapping', 'HasValueOrInitialCondition'})
```

simbiologyHandoffContract = 10×3 table

	Symbol	HasUnitMapping	HasValueOrInitialCondition
1	"Ea"	true	true
2	"Ks"	true	true
3	"R"	true	true
4	"S"	true	true
5	"T"	true	true
6	"Tref"	true	true
7	"X"	true	true
8	"Ypx"	true	true
9	"Yxs"	true	true
10	"muRef"	true	true

```
assert(all(simbiologyHandoffContract.HasUnitMapping) && ...
    all(simbiologyHandoffContract.HasValueOrInitialCondition), ...
    "Every exported SimBiology rate-law symbol must have a unit mapping and a value or initial condition.")
simbiologyExpressionStrings = string(arrayfun(@char, ...
    simbiologyExpressions, ...
    'UniformOutput', false));
simbiologyRateExpressions = table(["Specific growth rate"; "Biomass formation rate"; ...
    "Substrate uptake rate"; "Product formation rate"], ...
    simbiologyExpressionStrings, ...
    ["1/hour"; "gram/(liter*hour)"; "gram/(liter*hour)"; "gram/(liter*hour)"], ...
    'VariableNames', {'Quantity', 'Expression', 'ExpectedUnits'})
```

`simbiologyRateExpressions = 4×3 table`

	Quantity	Expression	ExpectedUnits
1	"Specific growth rate"	"(S*muRef*exp(-(Ea*(1/T - 1/Tref))/R))/(Ks + S)"	"1/hour"
2	"Biomass formation rate"	"(S*X*muRef*exp(-(Ea*(1/T - 1/Tref))/R))/(Ks + S)"	"gram/(liter*hour)"
3	"Substrate uptake rate"	"(S*X*muRef*exp(-(Ea*(1/T - 1/Tref))/R))/(Yxs*(Ks + S))"	"gram/(liter*hour)"
4	"Product formation rate"	"(S*X*Ypx*muRef*exp(-(Ea*(1/T - 1/Tref))/R))/(Ks + S)"	"gram/(liter*hour)"

```
simbiologyReactionRateHandoff = table(["null -> X"; "S -> null"; "null -> P"], ...
    simbiologyExpressionStrings(2:4), ...
    ["gram/(liter*hour)"; "gram/(liter*hour)"; "gram/(liter*hour)"], ...
    'VariableNames', {'Reaction', 'ReactionRate', 'ExpectedUnits'})
```

`simbiologyReactionRateHandoff = 3×3 table`

	Reaction	ReactionRate	ExpectedUnits
1	"null -> X"	"(S*X*muRef*exp(-(Ea*(1/T - 1/Tref))/R))/(Ks + S)"	"gram/(liter*hour)"
2	"S -> null"	"(S*X*muRef*exp(-(Ea*(1/T - 1/Tref))/R))/(Yxs*(Ks + S))"	"gram/(liter*hour)"
3	"null -> P"	"(S*X*Ypx*muRef*exp(-(Ea*(1/T - 1/Tref))/R))/(Ks + S)"	"gram/(liter*hour)"

The handoff contract checks the symbolic side of the boundary without turning this example into a SimBiology workflow. The `ReactionRate` column contains the exact strings assigned to SimBiology reaction `ReactionRate` properties in the next section. The specific growth rate is also shown because it is often reused as a custom kinetic-law expression.

17. Create a compact SimBiology handoff object when available

If SimBiology is installed, create a small model object and assign the generated symbolic expressions to reaction rates. This deliberately small section checks that the symbolic handoff artifact can be represented downstream without adding a full SimBiology simulation, optimization, or fitting workflow.

```
hasSimBiology = (exist('sbiomodel', 'file') == 2) && license('test', 'SimBiology');
if hasSimBiology
    simbiologyModel = sbiomodel('FedBatchRateLawReuse');
    simbiologyCompartment = addcompartment(simbiologyModel, 'bioreactor', initialStateVal(4));
    simbiologyCompartment.CapacityUnits = 'liter';
    addspecies(simbiologyCompartment, 'X', initialStateVal(1), 'InitialAmountUnits', 'gram/liter');
    addspecies(simbiologyCompartment, 'S', initialStateVal(2), 'InitialAmountUnits', 'gram/liter');
    addspecies(simbiologyCompartment, 'P', initialStateVal(3), 'InitialAmountUnits', 'gram/liter');
    biomassReaction = addreaction(simbiologyModel, 'null -> X');
    substrateReaction = addreaction(simbiologyModel, 'S -> null');
    productReaction = addreaction(simbiologyModel, 'null -> P');
    biomassReaction.ReactionRate = char(simbiologyReactionRateHandoff.ReactionRate(1));
    substrateReaction.ReactionRate = char(simbiologyReactionRateHandoff.ReactionRate(2));
    productReaction.ReactionRate = char(simbiologyReactionRateHandoff.ReactionRate(3));
    for parameterIndex = 1:height(simbiologyParameterHandoff)
        parameterObject = addparameter(simbiologyModel, char(simbiologyParameterHandoff.Name(parameterIndex)),
...
            simbiologyParameterHandoff.Value(parameterIndex));
        parameterObject.ValueUnits = char(simbiologyParameterHandoff.Unit(parameterIndex));
    end
    simbiologyReactionTable = table(string({simbiologyModel.Reactions.Name})', ...
        string({simbiologyModel.Reactions.Reaction})', string({simbiologyModel.Reactions.ReactionRate})', ...
        'VariableNames', {'Name', 'Reaction', 'ReactionRate'})
else
    simbiologyReactionTable = simbiologyReactionRateHandoff
end
```

`simbiologyReactionTable = 3×3 table`

	Name	Reaction	ReactionRate
1	"Reaction_1"	"null -> X"	"(S*X*muRef*exp(-(Ea*(1/T - 1/Tref))/R))/(Ks + S)"

	Name	Reaction	ReactionRate
2	"Reaction_2"	"S -> null"	"(S*X*muRef*exp(-(Ea*(1/T - 1/Tref))/R))/(Yxs*(Ks + S))"
3	"Reaction_3"	"null -> P"	"(S*X*Ypx*muRef*exp(-(Ea*(1/T - 1/Tref))/R))/(Ks + S)"

The same symbolic rate law is now visible as MATLAB generated functions, local state-space models, and SimBiology reaction-rate strings. One reviewed symbolic derivation feeds multiple downstream consumers.

18. Generated artifact summary

Keep generated files small, unit-free, and regenerated from this Live Script when kinetic assumptions or units change.

```
handoffSummary = table(["bioreactorRates.m"; "fedBatchBioreactorRHS.m"; "bioreactorJacobian.m"; ...
    "bioreactorProductivity.m"; "bioreactorProductivityGradient.m"], ...
    ["Rate law and temperature sensitivity"; "ODE RHS used by `ode15s` and reusable in Simulink or custom simulation workflows"; ...
    "State/input Jacobians used above for trajectory perturbation analysis and Control System Toolbox state-space models"; ...
    "Algebraic product formation rate used for local trade studies"; "Analytic substrate and temperature productivity sensitivities"], ...
    'VariableNames', {'GeneratedFile', 'DownstreamUse'})
```

handoffSummary = 5×2 table

	GeneratedFile	DownstreamUse
1	"bioreactorRates.m"	"Rate law and temperature sensitivity"
2	"fedBatchBioreactorRHS.m"	"ODE RHS used by `ode15s` and reusable in Simulink or custom simulation workflows"
3	"bioreactorJacobian.m"	"State/input Jacobians used above for trajectory perturbation analysis and Control System Toolbox state-space models"
4	"bioreactorProductivity.m"	"Algebraic product formation rate used for local trade studies"
5	"bioreactorProductivityGradient.m"	"Analytic substrate and temperature productivity sensitivities"